

Data Structures in C++20

A Review Booklet

Arrays • Linked Lists • Associative Containers • Binary Trees (BST, AVL, Red-Black)

Prepared with the C++ Standard Template Library (STL)

Introduction

This booklet is a compact review of the core data structures every C++ programmer should master. For each topic we first explain the underlying idea and its complexity trade-offs, and then show idiomatic C++20 code. Wherever the Standard Template Library (STL) already provides a well-tested, efficient implementation, we use it directly instead of writing the data structure from scratch — this is exactly what you should do in real projects.

We assume you are comfortable with basic C++ syntax, templates, and the idea of Big-O notation. Each section ends with a short complexity summary table for quick reference.

1. Static and Dynamic Arrays

1.1 Concept

An array is a contiguous block of memory holding elements of the same type, accessed by index in $O(1)$ time. C++ distinguishes between two flavors:

- Static array: its size is fixed at compile time. It typically lives on the stack (or is embedded in an object), so there is no heap allocation and no resizing. The C-style form is `T arr[N]`; the modern, safer form is `std::array<T, N>`.
- Dynamic array: its size can grow or shrink at run time. The elements still live in one contiguous heap buffer, but the container manages reallocation for you. In C++ this is `std::vector<T>`.

Both share $O(1)$ random access and good cache locality because elements are contiguous. The difference is entirely about size flexibility and where the memory lives.

`std::array<T, N>` — static array

`std::array` is a thin, zero-overhead wrapper around a C-style array that adds STL-container niceties: `size()`, `begin()/end()`, bounds-checked access with `at()`, and value semantics (it can be copied, compared, and passed by value safely).

```
#include <array>
#include <iostream>

int main() {
    // Size fixed at compile time (5). Lives on the stack.
    std::array<int, 5> grades{90, 85, 77, 92, 88};

    // Range-based for + structured access, both C++20-friendly
    for (const auto& g : grades)
        std::cout << g << ' ';
    std::cout << '\n';

    std::cout << "size: " << grades.size() << '\n';
    std::cout << "3rd element: " << grades.at(2) << '\n';

    // Compile-time error if you try grades.push_back(...):
    // std::array has no way to grow.
}
```

`std::vector<T>` — dynamic array

`std::vector` manages a heap buffer that it reallocates (typically doubling in size) whenever it runs out of room. Because reallocation is amortized, `push_back` is $O(1)$ amortized, but an individual call that triggers a reallocation is $O(n)$. If you know the final size ahead of time, call `reserve()` to avoid repeated reallocations.

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> scores;           // starts empty, size 0
    scores.reserve(4);                // pre-allocate capacity, avoid re-allocations

    scores.push_back(10);
}
```

```

scores.push_back(20);
scores.push_back(30);

scores.insert(scores.begin() + 1, 15); // O(n): shifts elements
scores.pop_back();                    // O(1): removes last element

for (auto s : scores)
    std::cout << s << ' ';
std::cout << '\n';

std::cout << "size: " << scores.size() << '\n';
std::cout << "capacity: " << scores.capacity() << '\n';
}

```

Note: Prefer `std::array` when the size is known at compile time and small (it avoids heap allocation entirely). Prefer `std::vector` as your default general-purpose sequence container — it is the C++ equivalent of a 'resizable array' found in other languages.

1.2 Complexity summary

Operation	<code>std::array</code>	<code>std::vector</code>
Random access [i]	O(1)	O(1)
Insert/erase at end	N/A (fixed size)	O(1) amortized
Insert/erase in middle	N/A (fixed size)	O(n)
Memory location	Stack (usually)	Heap
Resizable	No	Yes

2. Linked Lists

2.1 Concept

A linked list stores elements in separate nodes scattered in memory; each node holds a value and one or more pointers to neighboring nodes. Unlike arrays, elements are not contiguous, so random access is $O(n)$ — you must walk the list from a known point. In exchange, inserting or removing a node next to an iterator you already hold is $O(1)$, because no other elements need to shift.

- Singly linked list (forward list): each node points only to the next node. Traversal is one-directional. In C++: `std::forward_list<T>`.
- Doubly linked list: each node points to both the next and the previous node, allowing traversal in either direction and $O(1)$ removal given only an iterator to the node (no need to know its predecessor). In C++: `std::list<T>`.

`std::forward_list<T>` — singly linked list

`std::forward_list` is deliberately minimal (no `size()`, no `back()`) to guarantee it uses only one pointer per node, matching the classic singly linked list exactly. Insertion is done 'after' a position with `insert_after` / `emplace_after`, mirroring the fact that you cannot cheaply look backward.

```
#include <forward_list>
#include <iostream>

int main() {
    std::forward_list<int> lst{10, 20, 30};

    lst.push_front(5);           // O(1): add to the front

    auto it = lst.begin();       // points to 5
    lst.insert_after(it, 7);      // insert 7 right after 5 -> 5,7,10,20,30

    lst.remove(20);              // removes all nodes equal to 20

    for (int v : lst) std::cout << v << ' ';
    std::cout << '\n';
}
```

`std::list<T>` — doubly linked list

`std::list` supports bidirectional iteration, $O(1)$ insertion/removal anywhere given an iterator, and splicing whole ranges between lists without copying elements.

```
#include <list>
#include <iostream>

int main() {
    std::list<std::string> queue{"Alice", "Bob", "Carol"};

    queue.push_back("Dave");      // O(1)
    queue.push_front("Zoe");      // O(1)

    auto it = std::find(queue.begin(), queue.end(), "Bob");
    if (it != queue.end())
        queue.erase(it);         // O(1): no shifting needed
}
```

```

for (const auto& name : queue)
    std::cout << name << " -> ";
std::cout << "nullptr\n";
}

```

Note: In practice, `std::vector` often outperforms `std::list` even for insert/erase-heavy workloads, because contiguous memory is far more cache-friendly than pointer chasing. Reach for `std::list` or `std::forward_list` mainly when you need $O(1)$ splicing, stable iterators/pointers across insertions, or a strict singly linked structure.

2.2 Complexity summary

Operation	<code>std::forward_list</code>	<code>std::list</code>
Access by index	$O(n)$	$O(n)$
Insert/erase at known position	$O(1)$	$O(1)$
Insert/erase at front	$O(1)$	$O(1)$
Insert/erase at back	N/A	$O(1)$
Traversal direction	Forward only	Both directions

3. Key-Value Data Structures

3.1 Concept

Associative containers store elements so they can be looked up by key rather than by position. The STL offers two families, differing in how they organize keys internally:

- Ordered associative containers (`std::map`, `std::set`): implemented internally as a balanced binary search tree (in every mainstream implementation, a Red-Black tree). Keys are always kept in sorted order, so iterating the container visits keys from smallest to largest. Lookup, insertion, and removal are all $O(\log n)$.
- Unordered associative containers (`std::unordered_map`, `std::unordered_set`): implemented as a hash table. Keys have no defined iteration order, but lookup, insertion, and removal are $O(1)$ on average ($O(n)$ worst case, e.g. under many hash collisions).

`std::map` and `std::unordered_map` store key-value pairs (each element is a `std::pair<const Key, Value>`). `std::set` and `std::unordered_set` store only keys, with no separate mapped value — useful when you just need fast membership tests over a collection of unique items.

`std::map` — ordered key-value store

```
#include <map>
#include <iostream>

int main() {
    std::map<std::string, int> inventory;

    inventory["apples"] = 50;      // insert or update, O(log n)
    inventory["bananas"] = 30;
    inventory["cherries"] = 120;

    // Iteration visits keys in sorted (alphabetical) order
    for (const auto& [item, qty] : inventory) // C++17/20 structured bindings
        std::cout << item << ": " << qty << '\n';

    if (auto it = inventory.find("bananas"); it != inventory.end())
        std::cout << "Found bananas: " << it->second << '\n';

    inventory.erase("apples");    // O(log n)
}
```

`std::unordered_map` — hashed key-value store

```
#include <unordered_map>
#include <iostream>

int main() {
    std::unordered_map<std::string, int> word_count;

    for (const std::string& word : {"cat", "dog", "cat", "bird", "dog", "cat"})
        ++word_count[word];    // O(1) average per access

    // Iteration order is unspecified here
    for (const auto& [word, count] : word_count)
        std::cout << word << ": " << count << '\n';
}
```

std::set and std::unordered_set — key-only containers

```
#include <set>
#include <unordered_set>
#include <iostream>

int main() {
    std::set<int> ordered_ids{5, 1, 9, 3};          // stored sorted: 1 3 5 9
    std::unordered_set<int> fast_ids{5, 1, 9, 3};   // no defined order

    ordered_ids.insert(7);                          // O(log n)
    fast_ids.insert(7);                             // O(1) average

    std::cout << "contains 9? "
              << (ordered_ids.contains(9) ? "yes" : "no") // C++20 contains()
              << '\n';

    for (int id : ordered_ids) std::cout << id << ' ';
    std::cout << '\n';
}
```

Note: C++20 added the `contains()` member function to all associative containers, which is clearer than the older idiom of comparing `find()` against `end()`.

3.2 Choosing between ordered and unordered

- Need sorted iteration, range queries (`lower_bound/upper_bound`), or the best worst-case guarantees? Use `std::map` / `std::set`.
- Need the fastest average-case lookup and don't care about order? Use `std::unordered_map` / `std::unordered_set`.

3.3 Complexity summary

Operation	map / set	unordered_map / unordered_set
Search	$O(\log n)$	$O(1)$ average, $O(n)$ worst case
Insert	$O(\log n)$	$O(1)$ average, $O(n)$ worst case
Erase	$O(\log n)$	$O(1)$ average, $O(n)$ worst case
Iteration order	Sorted by key	Unspecified
Underlying structure	Red-Black tree	Hash table (buckets)

4. Binary Trees: BST, AVL, and Red-Black Trees

4.1 Binary Search Trees (BST)

A Binary Search Tree is a binary tree where, for every node, all keys in its left subtree are smaller and all keys in its right subtree are larger. This ordering lets you search, insert, and delete in $O(h)$ time, where h is the tree's height. The catch: if elements are inserted in sorted order, a naive BST degenerates into a linked list ($h = n$), making every operation $O(n)$.

4.2 AVL Trees

An AVL tree is a self-balancing BST. Every node tracks a balance factor (the height of its left subtree minus the height of its right subtree), which must stay in $\{-1, 0, 1\}$. After every insertion or deletion, the tree performs rotations (single or double, i.e. left, right, left-right, right-left) to restore this invariant. This keeps the height at $O(\log n)$ at all times, giving strictly $O(\log n)$ search, insert, and delete — even in the worst case.

AVL trees are more rigidly balanced than Red-Black trees, so lookups are slightly faster on average, but insertions/deletions can trigger more rotations, making them a better fit for read-heavy, write-light workloads.

4.3 Red-Black Trees

A Red-Black tree is also a self-balancing BST, but it relaxes the balance requirement compared to AVL. Each node is colored red or black, and the tree maintains these invariants:

- The root is black.
- Red nodes cannot have red children (no two consecutive red nodes on any path).
- Every path from a node to its descendant null leaves passes through the same number of black nodes (the 'black-height').

These rules guarantee the longest root-to-leaf path is never more than twice the shortest one, which bounds the height at $O(\log n)$. Re-balancing after insert/delete uses recoloring plus at most a small, constant number of rotations, which is cheaper on average than AVL's rebalancing. This is why Red-Black trees are the classic choice for general-purpose library containers that mix reads and writes.

Note: This is exactly what the STL uses under the hood: in every major implementation (libstdc++, libc++, MSVC STL), `std::map` and `std::set` are Red-Black trees, and `std::multimap` / `std::multiset` are the same structure allowing duplicate keys. The Standard does not mandate a Red-Black tree by name — it only mandates the $O(\log n)$ complexity guarantees — but a Red-Black tree is the structure every vendor has converged on.

Using the STL's built-in balanced BST

Because `std::set`/`std::map` already are balanced BSTs, you get BST behavior — and AVL/Red-Black-grade guarantees — for free, without writing a single rotation. The example below relies only on the tree ordering (no hashing), and shows the tree-specific operations that a hash table cannot offer: ordered traversal and range queries.

```
#include <set>
#include <iostream>

int main() {
    std::set<int> bst{50, 30, 70, 20, 40, 60, 80};
    // Internally a balanced (Red-Black) BST -> height stays  $O(\log n)$ 

    // In-order traversal of a BST always yields sorted order
```

```

std::cout << "in-order: ";
for (int key : bst) std::cout << key << ' ';
std::cout << '\n';

bst.insert(45); // O(log n): tree rebalances internally, transparently
bst.erase(70); // O(log n)

// BST-specific queries: smallest key >= 40, and > 40
auto lb = bst.lower_bound(40); // first element not less than 40
auto ub = bst.upper_bound(40); // first element greater than 40
std::cout << "lower_bound(40): " << *lb << '\n';
std::cout << "upper_bound(40): " << *ub << '\n';

// Range query: all keys in [30, 60]
std::cout << "range [30,60]: ";
for (auto it = bst.lower_bound(30); it != bst.upper_bound(60); ++it)
    std::cout << *it << ' ';
std::cout << '\n';
}

```

`std::map` behaves identically but stores a value alongside each key; the same `lower_bound/upper_bound` machinery works for ordered key-value range queries.

Why we don't hand-roll AVL / Red-Black trees here

Implementing rotation logic and re-balancing correctly (especially the multiple deletion cases in a Red-Black tree) is a classic exercise for learning tree invariants, but it is easy to get subtly wrong and is rarely something you should ship in production code. In real C++ codebases you reach for `std::map` / `std::set`, which give you a battle-tested, standard-compliant balanced BST. Understanding AVL and Red-Black trees conceptually — as we did above — is what lets you reason about the $O(\log n)$ guarantees those containers give you, and it is also common material for algorithms courses and technical interviews.

4.4 Complexity summary

Operation	BST (unbalanced)	AVL Tree	Red-Black Tree
Search	$O(n)$ worst case	$O(\log n)$	$O(\log n)$
Insert	$O(n)$ worst case	$O(\log n)$	$O(\log n)$
Delete	$O(n)$ worst case	$O(\log n)$	$O(\log n)$
Balancing strictness	None	Strict (height-balanced)	Looser (color rules)
Typical rotations per insert/delete	0	More (stricter rebalancing)	Fewer (cheaper rebalancing)
STL container using it	-	-	<code>std::map</code> / <code>std::set</code>

5. Overall Summary

The table below places every container discussed in this booklet side by side, so you can quickly pick the right tool for a given job.

Container	Underlying structure	Access	Insert/Erase	Ordered?
<code>std::array</code>	Contiguous, fixed-size	$O(1)$	N/A	By index
<code>std::vector</code>	Contiguous, resizable	$O(1)$	$O(1)$ amortized at end	By index
<code>std::forward_list</code>	Singly linked list	$O(n)$	$O(1)$ at known position	Insertion order
<code>std::list</code>	Doubly linked list	$O(n)$	$O(1)$ at known position	Insertion order
<code>std::map</code> / <code>std::set</code>	Red-Black tree	$O(\log n)$	$O(\log n)$	Sorted by key
<code>std::unordered_map</code> / <code>std::unordered_set</code>	Hash table	$O(1)$ avg.	$O(1)$ avg.	Unordered

Rule of thumb: default to `std::vector` for sequences and `std::unordered_map` for key-value lookups; move to `std::map`/`std::set` when you need sorted order or range queries; reach for `std::list` only when you need $O(1)$ splicing or iterator stability across insertions and removals.